

Virtualização de Redes

Data Plane Programming

Version: 1

Departamento de Informática
Universidade do Minho
Set 2025

João Fernandes Pereira

Assignment Scenario

A city is modernizing its communication infrastructure to support diverse services on a shared, programmable network. Instead of operating separate physical infrastructures, all applications—from emergency responders to public utilities—must coexist on the same backbone. To guarantee security, service differentiation, and accountability, the network employs **Service Function Chaining (SFC)**.

Different types of traffic require different treatments:

- **Critical emergency traffic** must be tightly controlled and always given priority, ensuring rapid delivery for mission-critical applications such as real-time video and control messages.
- **Controlled non-critical traffic** belongs to the same emergency services but does not require strict priority. It must be admitted under policy enforcement but forwarded with reduced guarantees, ensuring it does not interfere with critical flows.
- **Utility and public service traffic** originates from less critical applications such as maintenance and monitoring. This traffic is carried on a best-effort basis, with minimal processing overhead, and lower priority than emergency flows.

Your task in this assignment is to implement a dataplane solution in P4 that enforces these distinctions by steering packets through the appropriate service chains. Each chain will apply a specific combination of functions, ensuring that critical applications receive secure and prioritized treatment while background and utility traffic are safely supported on the same shared infrastructure.

Deliverables

Due Date: 02 Feb 2025, 23h59

You are required to submit **ONE** archive file with the following name format: **VR-25_26-studentNumbers.zip** (example: VR-25_26-53454_345454.zip).

Your archive file should contain the following:

- All Mininet scripts needed to create the topology.
- All the developed P4 code.
- All the defined rules for each of the P4 devices.
- A short report, in PDF format, with the following sections:
 - **Introduction:** Introduce the concepts of *Service Function Chaining (SFC)* and *dataplane programmability with P4*. Explain why SFC is relevant in shared infrastructures and how P4 enables its implementation.
 - **Implementation:** Describe your solution design. Include:
 - * **Custom SFC Header:** Explain your header format, fields, and field sizes. Describe how chains are selected at ingress, how the service index evolves along the chain, and how QoS class is encoded.

- * **Service Functions:** For each function (ACL, QoS marker, Counter/Egress), describe the logic and how it was realized in P4 tables and actions.
- **Tests & Results:** Present the experiments that validate your implementation. Demonstrate correct forwarding through each chain, enforcement of ACL policies, differentiation between fast (priority) and degraded (best effort) paths, and correct counting at the egress.
- **Conclusion:** Summarize the main challenges and lessons learned when implementing service chaining in P4.

Topology

The service function chains in this assignment will be deployed on a small programmable network topology composed of hosts, routers, a switch, and a server. Each router plays a specific role (ingress, ACL, QoS marker, counter/egress, or simple IPv4 forwarding), and packets are steered through different paths depending on the selected chain.

To ensure deterministic behavior, all nodes and interfaces use **predefined MAC addresses** rather than relying on Mininet's default random assignments. This guarantees consistency when installing forwarding rules and avoids problems with unexpected address changes. The formulas used to generate these MAC addresses for hosts, switches, routers, and the server are provided in Table 1.

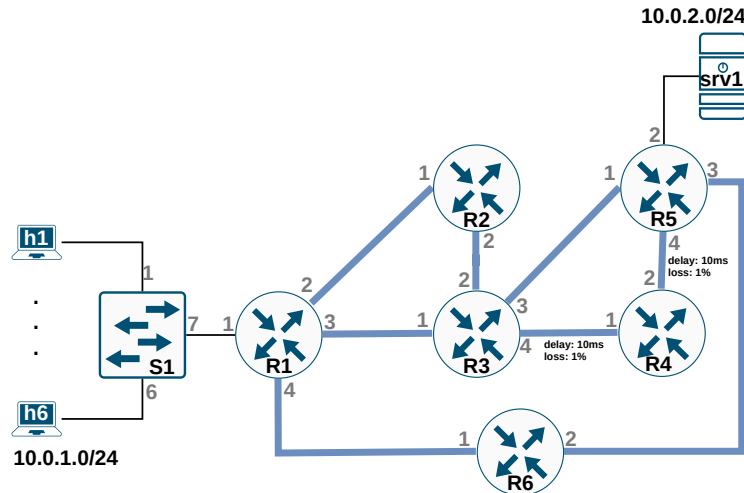


Figure 1: Network Topology.

Node type	MAC formula	Example
Host H_n	$aa:00:00:00:00:\{n\}$	$H_4 \rightarrow aa:00:00:00:00:04$
Switch S_1 port p	$ab:00:00:00:01:\{p\}$	$S_1\text{-eth3} \rightarrow ab:00:00:00:01:03$
Router R_r port p	$cc:00:00:00:\{r\}:\{p\}$	$R_3\text{-eth4} \rightarrow cc:00:00:00:03:04$
Server SVR1	$cb:00:00:00:00:01$	$SVR1 \rightarrow cb:00:00:00:00:01$

Table 1: Deterministic MAC addressing scheme (hex, zero-padded).

Note on degraded links. The topology includes two links (R3–R4 and R4–R5) that are intentionally configured with additional delay, reduced bandwidth, and a small packet loss rate. These links represent a **degraded path** in the network and are the main source of performance differences you will observe during your experiments. In contrast, the direct link R3–R5 remains unconstrained and serves as a fast path. The coexistence of both types of links in the topology is essential to demonstrate how the dataplane can steer traffic into paths with different quality characteristics.

Assignment Overview

In this assignment you will design and implement a programmable dataplane solution using the P4 language that enforces **Service Function Chains (SFCs)** over a shared network. The goal is to steer traffic into different chains, each applying a specific sequence of functions, according to the application's requirements.

The network topology provided to you represents a simplified city backbone, where different routers implement distinct service functions:

- Ingress classification of packets and encapsulation into the SFC namespace.
- Access Control List (ACL) enforcement for security.
- Quality of Service (QoS) marking and traffic steering into different paths.
- Transit through degraded links to model best-effort treatment.
- Egress decapsulation, forwarding, and packet counting for accountability.

Your dataplane must correctly:

1. Encapsulate traffic at ingress with the custom SFC header.
2. Forward packets along the chain defined for each traffic class.
3. Apply ACL policies, QoS marking, and counting as required by each chain.
4. Decapsulate traffic at egress and deliver it to the application server.

Return traffic from the server is not part of the service chains and should be forwarded using plain IPv4. This ensures the focus of the assignment remains on the client-to-server direction, where service chaining logic is applied.

Through this exercise you will gain practical experience in building custom headers, designing match-action tables, updating header fields across multiple hops, and ensuring consistency between dataplane logic and network configuration.

Service Chains Summary

The assignment requires the implementation of three distinct service chains, each combining a different set of functions:

- **Chain 1 – Critical Traffic** Packets are classified at ingress, verified by the ACL, marked as priority, and forwarded through the fast path before reaching the egress.
- **Chain 2 – Controlled Non-Critical Traffic** Packets are classified at ingress, verified by the ACL, marked as best-effort, and forwarded through the degraded path before reaching the egress.
- **Chain 3 – Utility/Public Traffic** Packets are classified at ingress, skip the ACL, are marked as best-effort, and traverse the degraded path before reaching the egress.

At the egress, all chains terminate with counting, header removal, and delivery to the application server.

Service Functions in the Topology

Each programmable router in the topology implements a specific service function. These functions are realized directly in P4 using simple match-action logic:

- **S1 – Layer-2 Switch** Forwards frames between the attached hosts and the ingress router. Implement a simple L2 forwarding table: match on the **Ethernet destination address** and set the correct egress port. Broadcast unknown destinations to all ports. (This ensures that hosts can reach the ingress router without manual intervention.)
- **R1 – Ingress** Classifies incoming traffic based on the **IPv4 source address** and the **IPv4 protocol field**. According to this classification, it encapsulates packets with the custom SFC header and assigns the chain

ID, service length, and initial service index. Return traffic from the server is forwarded using plain IPv4 only.

- **R2 – Access Control List (ACL)** Implements a Layer-3 ACL. Packets are matched on their **IPv4 destination address** and either permitted or dropped.
- **R3 – QoS Marker** Marks packets with a QoS class based on the **UDP destination port**; the QoS value is written into the SFC header (and/or DSCP). *Recompute the IPv4 header checksum after marking.*
- **R4 – Transit (Degraded Path, Pass-through)** Implements a minimal **cross-connect**: match on `standard_metadata.ingress_port` and set the **egress port** (e.g., `in=1→out=2`). No header changes. This keeps forwarding deterministic while the degraded behavior comes from the link parameters attached to R4.
- **R5 – Counter and Egress** Maintains per-chain counters (keyed by **chain ID and QoS class**), removes the SFC header, and forwards the packet to the server.
- **R6 – IPv4 Forwarding** Forwards return traffic from the server back to the ingress using plain IPv4 routing only.

Chains & Flows

Table 2 summarizes the mapping between traffic classes and the service chains they traverse. Each chain is identified and the ordered sequence of devices visited from ingress to egress.

Chain	UDP dst port	Device Sequence
Chain 1 – Critical	UDP : : 5555	R1 → R2 (ACL) → R3 (QoS=EF) → R5 (Counter/Egress)
Chain 2 – Controlled	UDP : : 6666	R1 → R2 (ACL) → R3 (QoS=BE) → R4 (Transit) → R5 (Counter/Egress)
Chain 3 – Utility/Public	UDP : : 80	R1 → R3 (QoS=BE) → R4 (Transit) → R5 (Counter/Egress)

Table 2: Traffic-to-chain mapping and device sequences.

Tasks – Overview

1. **Create the Topology:**
Use Mininet to create the topology shown in Figure 1. Ensure that all hosts, routers, and the server are connected as in the diagram. Apply the predefined IP and MAC addressing schemes provided in Table 1.
2. **Define the Custom SFC Header:**
Design the Service Function Chaining (SFC) header. Specify:
 - The fields included in the header (e.g., chain ID, QoS class, service length, service index).
 - The size (in bits) of each field.
 - The role of each field in steering packets through the chain.
3. **Implement the Ingress (R1) and Egress (R5) Devices:**
Develop the P4 logic for handling packets at the chain entry and exit points:
 - **Ingress Classification:** Match on IPv4 source address and protocol field to select the correct chain.
 - **SFC Header Construction:** Insert the custom SFC header with appropriate values for chain ID, QoS class, and service index.
 - **Egress Processing:** Remove the SFC header, update counters per chain/QoS, and forward the original packet to the server.
4. **Implement Intermediate Service Functions:**
Configure the routers that apply functions within the chains:
 - **R2 – L3 ACL:** Match on IPv4 destination address to permit or drop traffic.

- **R3 – QoS Marker:** Match on UDP destination port and set the QoS field in the SFC header (recompute checksum).
- **R4 – Transit (Degraded Path):** Implement port-to-port forwarding logic; degraded behavior is introduced by link configuration in Mininet.
- **R6 – IPv4 Forwarding:** Forward return traffic from the server to the ingress using plain IPv4.

5. Test and Validate the Chains:

Verify that each traffic class is forwarded along the correct chain:

- Send UDP traffic with destination ports 5555, 6666, and 80, and confirm it traverses Chain 1, Chain 2, and Chain 3 respectively.
- Check ACL enforcement at R2 by testing both allowed and blocked destinations.
- Compare performance of fast and degraded paths using `ping` (RTT differences) and `iperf` (throughput).
- Inspect counters at R5 to ensure they reflect the expected traffic per chain.